Sequence Diagrams

In part 5 of this lab, *we discussed* **ACTIVITY DIAGRAMS.** Now what sequence diagrams do, is take those and make them even more low level i.e. *mimic the actual way the system would work.*

- The core idea of an "Actor" and "System" remains
- The idea of both talking to each other

The only difference here is, *we're actually showing* the "**Talking Process**".

- In the last part you saw "METHODS", which are commands. That's the core of what we're doing here.
  - Instead of "*Search for Book*" the ACTOR (*Customer*) be bookSearch()
  - And the SYSTEM will take that message, and RETURN "**search result**"

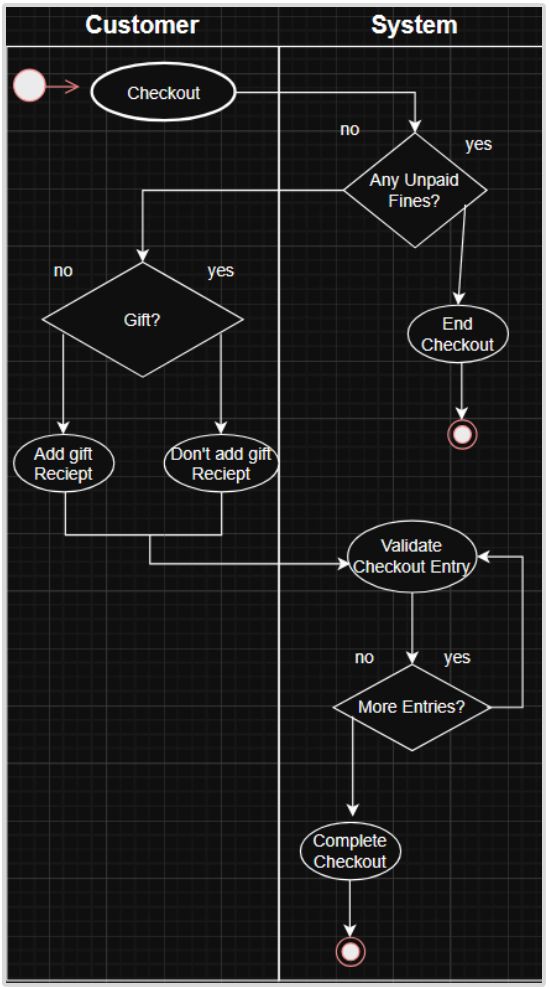
There are a couple QUIRKS about Sequence Diagrams; called **FRAMES**

- Certain messages may be special; it may LOOP, or just add one little thing and end the same way (called an OPTION), or result in a different ending entirely (called an ALTERNATE)
  - Think of a **loop** as, I may want to **validate a checkout** for EVERY *checkout entry in my shopping cart.*
  - Think of an **option** as, assume when I finish checkout I indicate it's a gift, *I still do my checkout; it's just slightly different.*
  - Think of **alternate** as, I do checkout but the system sees I have an unpaid fine; so I enter an ALTERNATE *path where I do NOT checkout*.

Let's use an EXISTING Activity Diagram, and convert it into a BASIC *sequence diagram.*

- We will use the activity diagram for the use case "*User Checkout*",
- **Analyze** to see what "METHODS" make sense, what "RESPONSES" make sense, as well as those **FRAMES.**

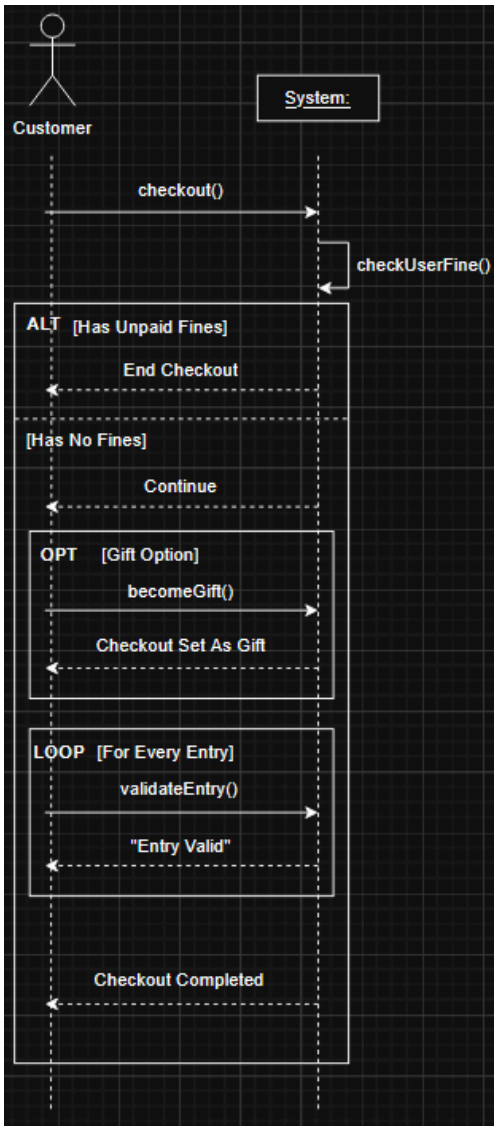
We'll see that on the next page.



In this diagram;

- As the customer we **send a message to the system** "I want to checkout"
  - Which is an ACTION, so a method; it would look like **checkout()**
- The system checks if the **user has any unpaid fines.**
  - Check is an ACTION, so a method; it would look like **checkUserFine()**
  - And it has 2 paths; *end checkout* **Or continue**, it's responding to the action "Checkout", with a response; which is just words "End Checkout."
    - Now see how "End Checkout" makes a DIFFERENT ending than if there wasn't a fine; *that's a sign that it's an* **ALTERNATE FRAME.**
- Assuming no fines, now we as the user are sending a NEW MESSAGE, indicating if we want it to be a **gift or not** (*either way we checkout*)
  - So this is another ACTION, i.e. a method, it would look like **becomeGift()**
  - And notice how I said, *either way we checkout*; because we end the same it's an **OPTION FRAME**
  - **System responds to message,** by telling customer it is now a gift or not
- Now we validate each checkout entry, *which is each part of the checkout* (10 of To Kill A Mockingbird, 2 of Kite Runner)
  - This **ACTION** has to be done for EVERY CHECKOUT ENTRY, *so it will ask if there's any left then;* and **LOOP back** if so, *thus a LOOP FRAME*.
  - The action or method will look like **validateEntry()**

- Then the system will END by returning “*Complete Checkout*”



The culmination of all that analysis turns into this.

- A solid line is used to SENT messages (METHODS), **matches our analysis**
- A dotted line is used to RETURN messages (Words), **matches our analysis**
- A BOX is used to symbolize a "FRAME"
  - Notice how the **FRAMES,** have the type (ALT, OPT, LOOP)
    - Then the scenario **[Has Unpaid Fines], [Gift Option], [Every Entry]**
  - Also notice how the rest of the diagram is INSIDE the ALT frame, that's because the ALTERNATE route, **of no fines is inside that frame.**
- *If you checkUserFine() you'll see it returns to ITSELF;* This is a flaw of the basic Sequence diagram.

That issue of a METHOD returning to itself is a flaw of a BASIC sequence diagram, a system isn't just ONE thing;

- It's multiple objects ***communicating with each other***
- *And,* just like the LAST part; *we want to use the* **THREE TIERED ARCHITECTURE,** to split up the "System" to more closely match the real implementation

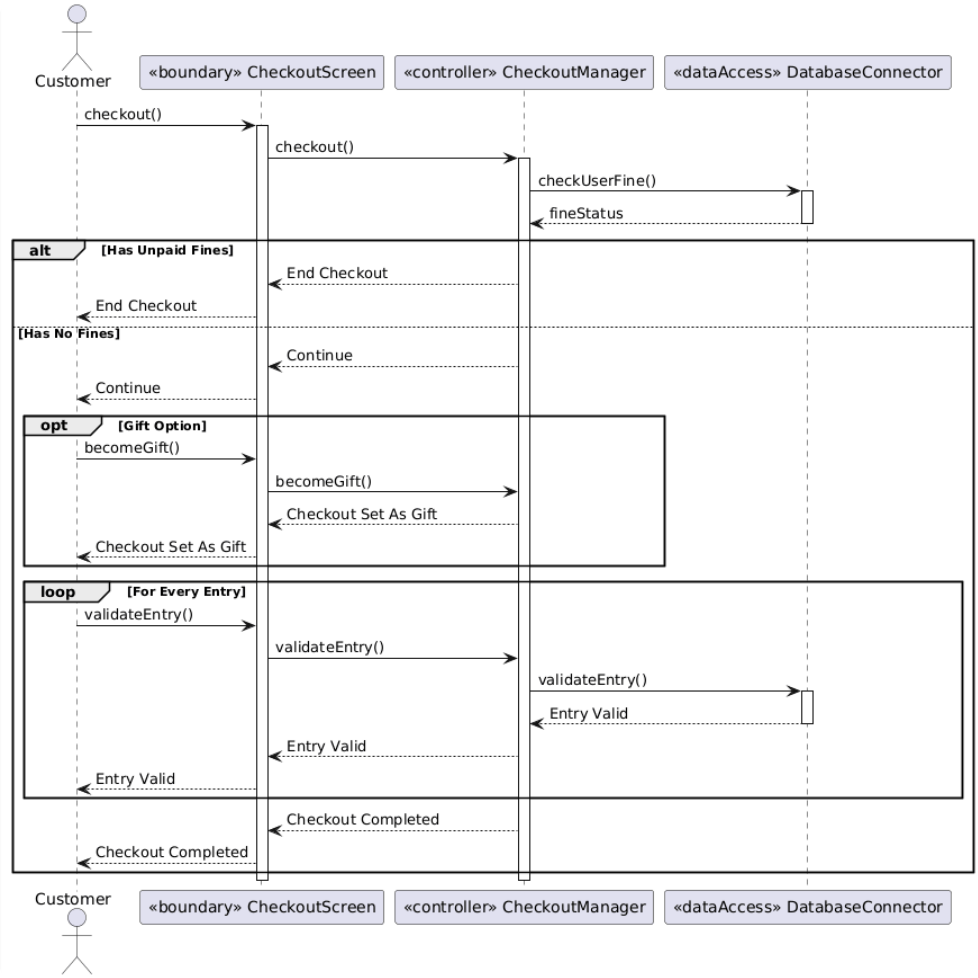
- Meaning, we’ll SPLIT the “System” into their specific objects, like the **<<dataAccess>>, <<controller>>, <<boundary>>**

For our **DETAILED** Sequence Diagram; Let's split the system into;

- <<Boundary>> checkoutForm
- <<Controller>> checkoutManager
- <<dataAccess>> userDatabase
  - We don't have "**<<entity>>**", *since* it's the Customer already.

Our METHODS, and RETURN messages *do not change*.

- It's just the **REAL part of the system does that specific METHOD/RETURN**
- **And an new WHITE VERTICAL BOX is used,** which is just saying further actions/return messages happen *after this first action*.
  - So the messages that the customer sent, *are TRULY* sent by the **BOUNDARY CLASS**,
  - The LOGIC is done by the **CONTROLLER CLASS**
  - And any logic that NEEDS data, **the CONTROLLER *grabs from* DATA-ACCESS**
  - *The messages always end up going back to the customer*.



That's actually it for BASIC SYSTEM DESIGN & SDLC, with this you can match ***up to a Intro University course on system design.***